Looking at the code snippet, the main issue is that PaymentProcessor is doing far too much on its own. Every payment method lives inside one long if elif chain, which means the class has to be opened up and edited every single time a new payment method gets added. That is a direct breach of the Open/Closed Principle, since a class is meant to be open for extension but closed for modification (Nesteruk, 2022, Part I). Sarcar (2022) shows the same issue with a payment example, where the class keeps needing edits every time a new method is introduced. The current code also mixes selection logic with payment logic in one method, which creates tight coupling, and readability only gets worse the more branches get added over time.

The Strategy Pattern fixes this by pulling each payment method out into its own class that all share a common interface. PaymentProcessor no longer needs to know how a payment actually gets processed, it just delegates to whichever strategy object it has been given (Nesteruk, 2022, Part II). Adding a new method like crypto later just means writing a new strategy class, with no changes needed to PaymentProcessor at all.

Python code

```
from abc import ABC, abstractmethod

class PaymentStrategy(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class CreditCardPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"Processing credit card payment of ${amount}")

class PayPalPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"Processing PayPal payment of ${amount}")

class BankTransferPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"Processing bank transfer of ${amount}")

class PaymentProcessor:
    def __init__(self, strategy: PaymentStrategy):
        self.strategy = strategy

    def set_strategy(self, strategy: PaymentStrategy):
        self.strategy = strategy

    def process_payment(self, amount):
        self.strategy.pay(amount)

if __name__ == "__main__":
    processor = PaymentProcessor(CreditCardPayment())
    processor.process_payment(150)

    processor.set_strategy(PayPalPayment())
```

```
processor.process_payment(75)
```

Including set_strategy() matters here because a real payment processor is rarely locked into one method for its whole lifetime, a customer might switch from card to PayPal partway through checkout, and the class should be able to accommodate that without being rebuilt from scratch.

The main benefit is flexibility, each strategy can be tested, changed or removed independently, and PaymentProcessor itself stays untouched no matter how many payment types get added later (Nesteruk, 2022, Part III). It is worth being honest about the limits of this though. For a system with only two or three payment types that are genuinely unlikely to ever change, introducing several extra classes and an interface is arguably more structure than the problem actually needs, and a simple if elif chain would do the job with less code to maintain. The pattern earns its value specifically when growth or change is expected, not as a default choice for every conditional statement in a codebase.

References

Nesteruk, D. (2022) *Design patterns in modern C++20: reusable approaches for object-oriented software design*. 2nd edn. California: Apress.

Sarcar, V. (2022) *Java Design Patterns: A Hands-On Experience with Real-World Examples*. 3rd edn. Berkeley, CA: Apress.